A BM REFERENCE

bm game API, volume 2: sound and music

Sound effects, notes, instruments, songs and riff patterns: every sound function of the cartridge API, with a first tutorial.

Contents

How sound works in bm 4

What it does 4

The synthesizer 4

The sound bank 5

Good to know 5

Caveats 5

Your first sound effect and your first music 6

Try it, step by step 6

Sound effects and music (the cartridge's bank) 8

What it does 8

Good to know 8

Example 8

Notes from the code 10

What it does 10

Caveats 10

Examples 10

Instruments, tone, room 12

What it does 12

The keys of `tone()` 13

Examples 15

Ready-made instruments 15

The console's kit 16

Bit depth 16

The bank: format and tools 17

What it does 17

Limits 17

The Sound editor 17

On the PC 17

Caveats 17

riff: music as patterns 19

What it does 19

Mini-notation 19

Functions 19

Live code 20

Tests 21

Open points 22

README

What this is. Volume 2 of the bm game API: everything a cartridge can do with sound and music, from `sfx(n)` to the riff patterns.

How to use it. Read "How sound works in bm", then follow the tutorial "Your first sound effect and your first music". After that, each chapter covers one area, with a table of every function and short examples.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

How sound works in bm

Audio goes out in **stereo** at 48 kHz. On the Pi it travels over HDMI, so you hear it from the monitor's speakers. On the RGB30 it comes from the headphones or the speaker. It is generated in an interrupt, so it costs nothing to your `_update` .

What it does

The synthesizer mixes 8 voices (numbered 0 to 7) in floating point, then runs a compressor and a soft limiter on the output. The player runs inside the same interrupt, every 64 samples (a grid of 1.3 ms): music and sound effects keep time whatever the frame rate of your game.

There are two ways to make sound, and you can use both together:

- **the sound bank** of the cartridge: sound effects and music made with the Sound editor of the Dev tab, played with `sfx(n)` and `music(n)` ;
- **notes** played by the code, one voice at a time: `note` , `slide` , `arp` ...

A third way sits on top of the notes: **riff** (see the chapter "riff: music as patterns"), which writes rhythms and melodies in a line and plays them on time.

The synthesizer

The synthesizer is **hi-fi**: waves without aliasing, exponential envelopes, a resonant filter, a room (reverb) and an echo, and a light compressor on the output. The **8-bit** voice of the first versions stays available: `retro(true)` , the `raw` key of `tone()` , or *Settings* > *Sound* > *Sound style*.

Waveforms (global names, usable in `note()`):

Wave	What it is
`SQUARE`	square, with `duty` (its width)
`TRIANGLE`	triangle
`SAW`	saw
`NOISE`	noise
`SINE`	sine
`METAL`	short metallic noise: cymbals, bells
`FM`	two sines, one bending the other: electric piano, bells, basses, brass
`PLUCK`	a plucked string: guitars, harps

Wave	What it is
`SUPERSAW`	three saws a little out of tune: pads, wide leads
`ORGAN`	four harmonics: organs, flutes
`SAMPLE`	a recorded sample: the bank's or one of the console's eight drums
`PINK`	a softer noise: wind, sea, hiss
`BROWN`	a softer noise too: wind, sea, hiss
`CRACKLE`	random clicks: an old record, a fire

Each voice has an ADSR envelope, a filter, a place between left and right, how much of it goes to the room, the echo and the chorus, and the effects of Strudel's superdough: bit crush, a lower rate, vowels, drive curves, tremolo, ducking (see `tone()`).

Pitches are in **Hz** (decimals too: `261.63`) or a **note name**: `"C4"` (middle C), `"A4"` (440 Hz), `"F#3"`, `"Bb2"`.

The sound bank

The bank is the **AUDIO** section of the `.bm` cartridge: sounds (instruments), sound effects, patterns and songs, and recorded samples. You make it with the Sound editor and it is saved inside the game. The chapter "The bank: format and tools" has the limits.

Good to know

- Eight voices, 0 to 7. Track `t` of a song pattern plays on voice `t`.
- Volumes are 0 to 255 for `note()` and 0 to 1 for `sfx()` and `play()`.
- The console's drum kit has eight drums made by the synthesizer when the console starts: nothing is recorded.

Caveats

- On the Pi, sound goes only over HDMI. An analog or I2S output is not there yet.
- **[TO FILL: the hi-fi synthesizer, the 8-bit style and the bit depth are not yet verified on the Pi and on the RGB30]**

Your first sound effect and your first music

You will make a sound with notes only (no bank needed), then a small piece of music with riff, then see how to use the bank.

Try it, step by step

1. Open a game project in the code editor, start from an empty `_update` . **[TO FILL: how to open a project and run it from the Dev tab, step by step]**
2. Write this in your code, then run the game:

```
function _update()
  if btnp(4) then
    note(0, 880, 60, SQUARE, 100)   -- a 60 ms "blip" on voice 0
  end
end
```

3. Press the **A** button (`btnp(4)` : keyboard Space, Z or J; DS4 cross; serial Space or `j`). You hear a short blip. If you hear nothing, check the general volume with `volume()` : it goes from 0 to 10.
4. Change `880` into `"C5"` . You hear a blip with the note's name.
5. Add a laser: a note that starts high and slides down. Replace the body of the `if` with:

```
note(1, 1300, 300, SQUARE, 70)   -- starts high...
slide(1, 300, 250)               -- ...and goes down in 250 ms
```

6. Add an explosion: noise that fades. Put these two lines inside the `if` :

```
envelope(2, 0, 60, 0, 30)        -- a sound that fades
note(2, 2500, 300, NOISE, 130)   -- with noise
```

7. Now music. Put this at the top of your file and call `R.update()` every frame:

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
end

function _update()
  R.update()                     -- every frame: the notes of the next moments into the queue
end
```

8. Run the game. You hear a drum loop: four kicks, a snare on the second half and eight hats a cycle.
9. To stop it, call `R.hush()` .
10. To play the cartridge's own sounds and songs, make them in the Sound editor of the Dev tab, which saves them inside the game. **[TO FILL: how to make a sound effect and a song in the Sound editor, step by step; see the Sound editor manual]**
11. Then play them from the code with `sfx(n)` and `music(n)` :

```
function _init()
  music(0)                          -- song 0, looping as decided in the editor
end
function _update()
  if btnp(4) then sfx(1) end        -- sound effect 1
end
```

TRY IT

Change the speed of the drums: `R.setcpm(30)` becomes `R.setcpm(40)`. Then add `R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid")` in `_init` and listen to the two together.

NOTE

The Sound editor's demo project `carts/sound/demo.json` has sound effects and three songs to listen to and copy.

BEWARE

`sfx(n)` returns `nil` when the cartridge has no bank, or when the number is not in the bank. A song and its effects need the bank, saved inside the game.

Sound effects and music (the cartridge's bank)

What it does

These functions play what you made in the Sound editor: sound effects with `sfx` and songs with `music`.

Function	Description
`sfx(n, [v], [semitones], [vol])`	plays sound effect `n` of the bank; without `v` it chooses a free voice, preferring those the music leaves empty. `semitones` transposes it (`sfx(0, nil, 12)`: an octave up), `vol` 0-1. Returns the voice, or `nil` (no bank, a number that is not there)
`sfx(-1, [v])`	stops the effect of voice `v`, or all
`sfxpos(v)`	the effect playing on voice `v` and its step, or `nil`
`music(n, [fade_ms], [pos])`	plays song `n` from the start (or from position `pos` of its sequence), fading in over `fade_ms`
`music(-1, [fade_ms])`	stops the music, fading it
`music()`	while it plays: song, position, step, pattern (to keep in time); otherwise `nil`
`tempo(x)`	the music goes `x` times faster (1 = as written): speed up when the game gets hard
`mute(track, [on])`	turns off (`on`, the default) or back on a track of the music: layers that come and go
`volume([level])`	the general volume 0-10 (with `level` it changes it). It is the console's: it holds for every game and stays in `bm/config.txt`

Good to know

- Track `t` of a pattern plays on voice `t`.
- While a sound effect uses a voice, the music's track on that voice is silent.
- A bank with a song that uses tracks 0-5 leaves voices 6 and 7 free for the effects.
- `volume()` is the console's volume, not the game's: it holds for every game.

Example

```
function _init()
  music(0)                        -- song 0, looping as decided in the editor
end
function _update()
  if btnp(4) then sfx(1) end      -- jump
  if got_coin then sfx(0) end
  if boss then tempo(1.2) end     -- faster
  if btnp(8) then music(-1, 500) end
end
```

COPY AND PASTE

```
sfx(0, nil, 12)      -- sound effect 0, an octave up
music(1, 500)        -- song 1, fading in over 500 ms
mute(2)              -- turn off track 2 of the music
mute(2, false)       -- and back on
```

Notes from the code

What it does

These functions play a note on one voice. The waveform and the volume stay those of the voice's last note, so giving them once is enough.

Function	Description
`note(v, hz, [ms], [wave], [vol])`	plays a note on voice `v` (the envelope starts again); with `ms` it ends by itself, without it it stays on until `noteoff(v)`. `hz` in Hz or a name (`"C4"`). `vol` 0-255 (default 128)
`noteoff(v)`	releases the note (the release phase starts)
`freq(v, hz)`	changes the pitch without starting again
`slide(v, hz, [ms])`	the note slides to `hz` in `ms` (default 100): glissandos, sirens, lasers
`vibrato(v, [semitones], [hz])`	a vibrato `semitones` wide (for example 0.3) at `hz` swings a second (default 6); `vibrato(v)` takes it away
`arp(v, chord, [ms])`	the note runs through a chord, `ms` per note (default 50): `"major"` , `"minor"` , `"maj7"` , `"min7"` , `"7"` , `"sus2"` , `"sus4"` , `"dim"` , `"aug"` , `"power"` , `"octave"` , or a table of semitones (`{0, 4, 7, 12}`); `arp(v)` takes it away
`hz(note)`	the frequency of a note: a MIDI number (60 = middle C, 69 = A 440) or a name (`hz("A4")` = 440)
`envelope(v, a, d, s, r)`	the voice's envelope: attack, decay and release are times 0-255 (0 = instant, 255 = 2 s), sustain is a level 0-255. Default `1, 0, 255, 10`
`duty(v, d)`	the width of the square wave, 0-255 (128 = 50%; 32-64 sounds more "nasal")
`playing(v)`	`true` while the voice sounds (release included) or an effect / the music holds it
`apu(v, reg, [value])`	reads or writes a raw register of the voice (32 bytes per voice, 0-31: `src/audio/synth.h` ; register 10 is the 1/256 of Hz, from 11 the tone: filter, place, room, echo, LFO)

Caveats

- At the start and at the end of the cartridge the voices go off and back to their defaults.
- Several loud voices together add up: a limiter keeps them under the top, but keep the volumes around 100-130.

Examples

```
note(0, 880, 60, SQUARE, 100)          -- a 60 ms "blip"
note(0, "C5", 60, SQUARE, 100)         -- the same with the note's name
envelope(2, 0, 60, 0, 30)              -- an explosion that fades...
note(2, 2500, 300, NOISE, 130)         -- ...with noise
note(1, 1300, 300, SQUARE, 70)         -- laser: starts high...
slide(1, 300, 250)                     -- ...and goes down
note(3, "C4", 600, SQUARE, 90)         -- an arpeggiated major chord
arp(3, "major", 40)
```

Pong, Snake and Star Shooter in `carts/` use notes (a 10-line `jingle` function for the tunes; now there is `lib.jingle` in bmlib).

TRY IT

Play a held note and release it: `note(0, "A4")` starts it and keeps it on, `vibrato(0, 0.3)` makes it swing, `noteoff(0)` releases it.

Instruments, tone, room

What it does

`tone()` gives a voice a whole sound (an instrument, a filter, effects) for the `note()` s that follow. `play()` plays an instrument as the music plays it. `reverb` , `echo` and `chorus` set the room, the echo and the chorus that every voice sends to.

Function	Description
`tone(v, sound)`	the voice's tone for the `note()` s that follow: the name of a **ready-made instrument** (`"epiano"` , `"pluck"` , `"pad"` , `"kick"` ... `instruments()` lists them) or a table in plain units (below). `tone(v)` goes back to the square of a fresh voice
`play([v], sound, [note], [ms], [vol])`	plays an instrument as the music plays it, with its pitch envelope and vibrato (the kick that falls, the laser): a name, a table as `tone()` 's (plus `pitch` semitones to start from, `ptime` ms to get there, `vib` cents, `vibhz` , `detune` cents) or a sound of the bank (a number; a name is first a sound of the bank with that name, then a ready-made instrument, then a drum of the kit: `"bd"` , `"sd"` , `"hh"` , `"oh"` , `"cp"` , `"cb"`). `"NAME:N"` is a sample's sound on its N-th sample after its own (`"kit:2"` the kit's third drum, the hi-hat; for a sound of the bank the bank's samples; wrapping). `note` : a MIDI number or a name (`"C4"` , default 60), `ms` 0 = held until `noteoff` , `vol` 0-1; without `v` a free voice. Returns the voice
`instruments([kind])`	the ready-made instruments: `{ {name=, kind=, about=}, ... }` ; `kind` : `"drum"` , `"bass"` , `"keys"` , `"pad"` , `"pluck"` , `"lead"` , `"fx"`
`instrument(name)`	an instrument with all its values, as bm Sound keeps a sound (`wave` , `a` , `d` , `s` , `r` , `pitch` , `tone` = the tone's 21 bytes...), or `nil` ; also a drum of the kit (`"sd"`) or `"kit:3"`
`reverb([size], [damp], [wet])`	the room the voices play in (0-1: a small room to a hall, bright to dull, how much is heard); returns the three. Each voice sends to it what its `reverb` says (a little by default: 0.16)
`echo([ms], [feedback], [wet])`	the ping-pong echo (left, right): the time between repeats (680 ms at most), how much comes back (0-0.95), how much is heard; returns the three. What goes in is each voice's `echo`
`chorus([rate_hz], [depth_ms], [wet])`	the chorus (two delays that swing, left and right: width for pads and keys): how fast it swings (0.05-5 Hz, default 0.8), how far (0-8 ms, default 2.5), how much is heard (0-1, default 1); returns the three. What goes in is each voice's `chorus`

Function	Description
`audio_depth()`	`bits, out` : the bits of each sample the console sends (16, 24 or 32: `sound_depth` in `bm/config.txt` , *Settings* > *Sound* > *Bit depth*, 24 by default) and what the output asks for now (0: none). A game reads it, it does not change it
`audio_samples(["kit"])`	the samples of the cartridge's bank (entry `n + 1` is the one `sample = n` plays) or, with `"kit"` , the kit's eight drums: `{ {name=, frames=, rate=, channels=, root=, fine=, loop=, loop_start=, loop_end=, peak=, meter=}, ... }` . `rate` Hz, `root` MIDI note, `fine` cents, `loop` 0 none, 1 forward, 2 ping-pong (`loop_start` , `loop_end` in frames), `peak` 0-1 of full scale, `meter` 32 peaks 0-1 from its start to its end
`retro([on])`	every voice **8-bit** as in the first versions (naive waves, straight envelopes, no room nor echo) while the game runs; returns whether they are (Settings can ask for it too)
`audio_time()`	the sound's clock in seconds (it moves with the samples played)
`play_at(t, sound, [note], [ms], [vol], [tag])`	a note at the time `t` of `audio_time()` : the sound's interrupt starts it, within 1.3 ms, whatever the frame rate. `sound` as `play()` 's, `ms` how long it is held (250 by default), `vol` 0-1, `tag` 1-255 (a group for `play_cancel` , 1 by default). The voice is chosen when it starts (a free one, a tail, the oldest note of `play_at` ; never one of a song or of an effect). `true` , or `false` when the queue (160 notes) is full. riff uses it
`play_cancel([tag])`	forgets the notes of `play_at` waiting with that `tag` (none: all) and releases those that sound
`play_voices([v, ...])`	the voices `play_at` may take (none: all 8); returns how many notes wait

The keys of `tone()`

All the keys are optional; the missing ones stay as they are.

Key	Value
`preset`	a ready-made instrument to start from
`wave`	`"square"` , `"triangle"` , `"saw"` , `"noise"` , `"sine"` , `"metal"` , `"fm"` , `"pluck"` , `"supersaw"` , `"organ"` , `"sample"` , `"pink"` , `"brown"` , `"crackle"`
`vol` , `duty` , `sustain`	0-1
`attack` , `decay` , `release`	ms (up to 2000)

Key	Value
`cutoff`	the **filter**: Hz (0 = no filter)
`res` , `filter`	resonance 0-1; `"lp"` low pass, `"bp"` band pass, `"hp"` high pass, `"notch"`
`keytrack`	`true` : the cutoff follows the note (at middle C it is the one given)
`fenv` , `fdecay`	the filter's envelope: octaves that open it (negative too), ms to close again
`lfo` , `wah` , `pwm`	a slow wobble (Hz) that moves the filter (`wah` octaves) and the square's width (`pwm` 0-1)
`drive` , `noise`	soft saturation before the filter, white noise added (0-1)
`pan`	-1 left, 0 middle, 1 right
`reverb` , `echo`	how much the voice sends to the room and to the echo (0-1)
`ratio` , `depth` , `mdecay` , `feedback`	FM: the modulator's ratio (1, 2, 3.5...), depth (radians, 0-8), in how many ms it fades, feedback 0-1
`bright` , `ring`	PLUCK: how bright the string is and how long it rings (0-1)
`spread`	SUPERSAW: how out of tune the three saws are (0-1)
`bars`	ORGAN: the four drawbars `{8, 6, 3, 2}` (0-15)
`sample` , `begin` , `reverse`	SAMPLE: which sample (a number of the bank's, a name of the bank's, a drum of the kit `"bd"` `"sd"` `"hh"` `"oh"` `"cp"` `"rim"` `"tom"` `"cb"` , or `"kit:N"`), where it starts (0-1 of its length), `true` backwards. The note against the sample's root sets its speed
`density`	CRACKLE: how many clicks (0-1)
`color`	the colour of the `noise` mixed in: `"white"` , `"pink"` , `"brown"` , `"crackle"`
`curve`	the `drive` 's curve: `"soft"` (as before), `"hard"` , `"fold"` , `"sine"` , `"asym"` (even harmonics, a tube), `"cubic"`
`vowel`	the formants of a vowel after the filter: `"a"` , `"e"` , `"i"` , `"o"` , `"u"` , `""` none
`crush` , `coarse`	bit crush: the bits kept (1-15; 0 none); the rate divided (1-16: each value held that many samples)
`trem`	tremolo: the `lfo` on the volume, how deep (0-1)
`duck`	ducking: this voice pushes the others down while it sounds (0-1): a kick that pumps a pad, an effect that lowers the music

Key	Value
`chorus`	how much the voice sends to the chorus (0-1; `chorus()` sets it)
`raw`	`true` : this voice is 8-bit (the effects above are ignored, as in `retro`)

Every new key at 0 (or missing) is the sound as it was. `play()` and `play_at()` take the same keys in their table.

Examples

```
tone(0, "epiano")                                  -- an electric piano for note()
note(0, "E4", 400)
tone(1, { preset = "bass", cutoff = 300, res = 0.7 })   -- a darker, ringing bass
play(nil, "kick", "C2")                            -- the kick, on a free voice
play(nil, { wave = "noise", cutoff = 900, fenv = -2, fdecay = 300, decay = 400, sustain = 0 }, "C3")
play(nil, "sd")                                    -- the kit's snare (a sample)
play(nil, { preset = "kit", sample = "hh", reverse = true })  -- the hi-hat backwards
tone(2, { preset = "lead", crush = 4, coarse = 3 }) -- a broken lead, 4 bits
tone(3, { preset = "pad", vowel = "o", chorus = 0.6 })  -- a choir singing "oh"
play(nil, { preset = "kick", duck = 0.8 }, "C2")    -- the kick lowers everything else
reverb(0.8, 0.5, 1)                                -- a cathedral (Yharnam)
echo(375, 0.4, 1)                                  -- an echo on the beat at 80 BPM
chorus(0.5, 4, 1)                                  -- a slower, wider chorus
retro(true)                                        -- the 8-bit sound of before
```

Ready-made instruments

There are 50 (`src/audio/presets.c`):

Kind	Instruments
drums	`kick` , `punch` , `snare` , `clap` , `hat` , `openhat` , `tom` , `rim` , `crash` , `cowbell` , `shaker` ; in 8 bits `chipkick` , `chipsnr` , `chiphat` ; `kit` (the kit's drums as samples: `"kit:0"` ... `"kit:7"`); `pump` (the kit's kick ducking the rest: a sidechain pump)
basses	`bass` , `acid` , `sub` , `fmbass` , `pickbass` , `bitbass` (crushed to 4 bits)
keys	`epiano` , `organ` , `bell` , `marimba` , `glock` , `rhodes` (with tremolo and chorus)
pads	`pad` , `strings` , `warm` , `glass` , `lush` (supersaw in the chorus), `choir` (singing "aah")
strings	`pluck` , `guitar` , `harp`
leads	`lead` , `sawlead` , `flute` , `brass` , `triangle` , `chip` , `chiptri` , `solo` (warm drive, vibrato, echo)
effects	`laser` , `blip` , `boom` , `wind` , `vinyl` (a record's crackle)

The same instruments are in bm Sound's *Instrument...* menu.

The console's kit

Eight drums made by the synthesizer when the console starts (nothing recorded), played as samples: `bd` kick, `sd` snare, `hh` closed and `oh` open hi-hat, `cp` clap, `rim` , `tom` , `cb` cowbell (`"kit:0"` ... `"kit:7"` in this order).

- `play(nil, "cp")` and `tone(v, { wave = "sample", sample = "oh" })` play them.
- `rim` and `tom` by name are the synthesizer's instruments; the kit's are `"kit:5"` and `"kit:6"` .
- In the 8-bit mode they still sound like drums (8 bits, the nearest frame).

Bit depth

The mix is in floats and is rounded once, with dither, to the depth the output carries: `sound_depth=16|24|32` in `bm/config.txt` (*Settings* > *Sound* > *Bit depth*, 24 by default). The Pi's HDMI and the RGB30's codec convert at most 24 bits; `16` is the old path exactly. `audio_depth()` tells a game which it is.

BEWARE

[TO FILL: the 24-bit and 32-bit output has not been tried on the RGB30 console yet; if 24 sounds wrong there, `sound_depth=16` is the old path exactly]

TRY IT

Call `tone(0, "epiano")` and `note(0, "E4", 400)` . Then `reverb(0.8, 0.5, 1)` and play the note again: you hear the same note in a big room.

The bank: format and tools

What it does

The bank is the **AUDIO** section of the `.bm` (format in `src/audio/player.h`, version 2, or 3 with samples; version 1 is still read).

Limits

- up to 32 sounds (instruments, each with its tone: filter, place, room, echo, LFO, effects);
- 64 sound effects;
- 64 patterns;
- 8 songs (each with the echo on the beat, in steps, and the room's size);
- up to 64 **samples**: 8, 16, 24, 32-bit or float, mono or stereo, any rate, with a root note and a loop (forward or ping-pong); 16 MB a bank, about 174 s of mono at 48 kHz.

The Sound editor

The bank is made with the **Sound editor** (Dev tab), which opens a game and saves its sounds right inside it. Its menu's *Samples...* lists the bank's samples and the kit's with their length and peak, and puts one into the sound (the SAMPLE wave, with SAMPLE and START among its settings). The menu's footer shows the output's depth (`output 24-bit`).

On the PC

Command	What it does
`scripts/bmaudio.py unpack game.bm -o sounds.json`	extracts the bank as readable JSON (the samples as WAV files beside it)
`mkbm.py --audio sounds.json`	puts the JSON back in a cartridge
`make wav BANK=sounds.json SONG=0`	plays the bank into a WAV (`BITS=24` , `32` or `f32` for a deeper file; 16 by default)
`bmhost --wav-bits 24`	records a game's sound at 24 bits

A WAV file goes into a bank from the JSON, and a sound plays it by name:

COPY AND PASTE

```
"samples": [{"name": "voice", "wav": "voice.wav", "root": "C4", "loop": "fwd"}]
"wave": "sample", "sample": "voice"
```

Caveats

- A sound pack in `/bm/sounds` (a `.bm` with only AUDIO) can still be read; bm Sound saves new ones as `.bms` files in `/bm/lib` .
- bm Sound's **Import from...** copies the sample of a SAMPLE sound with the sound and renumbers the sound's `MOD1` . A sample with the same bytes is not copied twice. The kit's drums need no copy. A bank holds at most 64 samples.

- bm Sound's **Import WAV or MIDI...** (Ctrl+W) reads `.wav` and `.mid` files from `/docs` on the SD card with `doc_list` and `doc_read` : the tool needs no permission. A WAV becomes a bank sample (8 to 32-bit integers and 32 or 64-bit floats, mono or stereo, up to 4 MB, over 48000 Hz brought down to 48000 Hz, the root note and loop of a `smpl` chunk kept). A MIDI file becomes a song of 64-step patterns, with the drum channel on the kit's drums.

riff: music as patterns

What it does

`local R = require "riff"` : rhythms and melodies written in a line, as in TidalCycles and Strudel, played on time by the sound's interrupt (`play_at`) with the ready-made instruments and the bank's sounds by name. Time is counted in **cycles** (a bar of four beats; 2 s at first). The whole pattern language is in the riff manual.

```
local R = require "riff"

function _init()
  R.setcpm(30)
  R.play("drums", R.s "kick*4, ~ snare, hat*8")
  R.play("bass", R.note "<c2 a1 f1 g1>" :s "acid" :lpf(R.sine:range(300, 1800):slow(4)))
end

function _update()
  R.update()                      -- every frame: the notes of the next moments into the queue
end
```

Mini-notation

Mini-notation	What it does
`a b c d`	a sequence in the cycle
`~` `-`	a rest
`[a b]`	a group in one part
`a, b`	together
`<a b>`	one a cycle
`a*2` , `a/2`	faster, slower (also `a*<2 4>`)
`a!3` , `a@3` , `a _ _`	repeated, longer
`a?` , `a?0.3`	at random
`a(3,8,2)`	a Euclidean rhythm
`{a b c}%4`	polymeter
`a | b`	one of the two, each cycle
`kick:2`	a variant (two semitones up)
`kit:2` , `bd sd hh`	the kit's drum 2 (samples, at their own speed); the kit's drums by name, as in Strudel

Functions

Function	Description
`R.s(p)` , `R.note(p)` , `R.n(p)` , `R.chord(p)`	instruments, notes (`c4` = 60, octave 3 when missing), degrees of a scale, chords (`"<Am F C G7>"`)
`R.seq` , `R.cat` , `R.stack` , `R.timecat` , `R.arrange` , `R.run(n)`	in sequence, a cycle each, together, with weights, sections of several cycles, `0..n-1`
`R.sine` , `R.cosine` , `R.saw` , `R.tri` , `R.square` , `R.rand` , `R.perlin` , `R.irand(n)` , `R.choose(...)`	signals 0..1
`:fast` , `:slow` , `:early` , `:late` , `:rev` , `:ply` , `:iter` , `:palindrome`	time
`:every(n, f)` , `:lastOf` , `:sometimes(f)` , `:often` , `:rarely` , `:degradeBy(x)` , `:someCycles`	changes now and then, at random
`:euclid(k, n, r)` , `:struct(p)` , `:mask(p)` , `:segment(n)` , `:chunk(n, f)` , `:linger(x)` , `:swing(n)`	structure
`:off(t, f)` , `:superimpose(f)` , `:layer(...)` , `:jux(f)`	layers, left and right
`:add` , `:sub` , `:mul` , `+` , `:transpose` , `:scale("C:minor")` , `:arp("updown")` , `:range(a, b)`	notes and numbers
`:s` , `:gain` , `:legato` , `:lpf` , `:hpf` , `:bpf` , `:res` , `:pan` , `:room` , `:delay` , `:attack` , `:decay` , `:sustain` , `:release` , `:shape` , `:vib` , `:fm` , `:raw` , `:crush` , `:coarse` , `:vowel` , `:chorus` , `:trem` , `:duck` , `:begin` , `:tone{...}`	the sound of each note (the units of `tone()` ; `:tone{curve = "fold", color = "pink"}` for the rest)
`R.play(name, p)` , `R.stop(name)` , `R.hush()` , `R.update()`	play (with a name), stop, to call in `_update`
`R.setcps(x)` , `R.setcpm(x)` , `R.bpm(x)`	the speed
`R.code(text)`	live code: riff's functions are globals, every global given a pattern plays under its name, the earlier ones not named again stop; `true` or `nil` and the error
`R.bake(p, {cycles=, steps=16})` , `R.piece(t)`	the pattern as a piece for the bank (the shape of `ai.music`), a piece of `ai.music` as a pattern
`R.playing()` , `R.get([name])` , `R.errors()` , `R.active()` , `R.voices(v, ...)`	the names playing, their patterns, the errors, where the notes sounding are written, the voices it may use

Live code

```
R.code [[
setcpm(28)
local verse = chord "<Am F C G>"
pad   = verse :s "pad" :room(.6) :gain(.7)
arp   = verse :s "pluck" :arp("updown") :fast(4)
drums = s "kick ~ ~ kick, ~ snare, hat*8?" :every(4, fast(2))
]]
```

```
R.code [[
beat  = s "bd ~ [~ bd] ~, ~ cp, hh*8" :crush(8)        -- the kit's samples, 8 bits
vox   = note "<a3 f3>" :s "choir" :vowel("o") :chorus(.5)
]]
```

In **bm Code**, Ctrl+Enter plays the code (the tab, or the `riff.code [[ ]]` block under the cursor) and lights up the words of the notes sounding; Ctrl+. stops it. In **bm Sound**, F7 (*Riff...*) plays a line and Ctrl+Enter puts it into the bank as a song.

Tests

`make test-riff`, and the QEMU tests `test_riff_code` and `test_sound_riff`.

Open points

- **How sound works in bm**: the hi-fi synthesizer, the 8-bit style and the bit depth are not yet verified on the Pi and on the RGB30
- **Your first sound effect and your first music**: how to open a project and run it from the Dev tab, step by step
- **Your first sound effect and your first music**: how to make a sound effect and a song in the Sound editor, step by step; see the Sound editor manual
- **Instruments, tone, room**: the 24-bit and 32-bit output has not been tried on the RGB30 console yet; if 24 sounds wrong there, `sound_depth=16` is the old path exactly